Student 38 **SHEKAR DHARANISH** 192472152RD 5 / 5 submitted

Q1. ATM Withdrawal – Exception Handling Debugging

CODE

```
import java.io.*;
class Main{
    public static void main(String[] args) {
        try{
            BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
            String[]x=br.readLine().split("");
            int balance=Integer.parseInt(x[0]);
            int amount=Integer.parseInt(x[1]);
            if(amount<=0)
                throw new IllegalArgumentException("Insufficient balance");
            if(amount>balance)
                throw new ArithmeticException("Insufficient balance");
            System.out.println("withdrawl successful");
            System.out.println("Remaining balance="+(balance-amount));
        }catch(ArithmeticException e) {
            System.out.println(e.getMessage());
        }catch(NumberFormatException|ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid input");
        }catch(Exception e) {
            System.out.println("Transaction failed");
        }finally{
            System.out.println("Transaction completed");
        }
    }
}
```

OUTPUT

```
Transaction failed
Transaction completed
```

Q2. Hospital Registration – NullPointerException Debugging

CODE

```
import java.util.Scanner;
class PatientRegistration{
    public static void main(String[] args){
        Scanner sc= new Scanner(System.in);
        try{
            String name= sc.nextLine();
            if(name.equals("NULL")||name.isEmpty())name="unknown";
            int age=Integer.parseInt(sc.nextLine());
            System.out.print("Patient Name:"+name.toUpperCase());
            if (age<0)
                System.out.print("Adult patient");
            else
                System.out.print("Minor patient");
        }catch(Exception e){
            System.out.print("Invalid age input");
        }finally{
            System.out.print("Registration completed:");

        }
    }
}
```

INPUT

```
dharanish
17
```

OUTPUT

```
Patient Name:DHARANISHMinor patientRegistration completed:
```

Q3. E-Commerce Product Search – Array Exception Debugging

CODE

```
import java.io.*;
class Main{
    public static void main(String[] args){
        try{
            BufferedReader b=new BufferedReader(new InputStreamReader(System.in));
            String[] p={"Laptop","Mobile","Tablet","Headphone","Keybopard"};
            int n=Integer.parseInt(b.readLine());
            if(n<1||n>5)throw new Exception();
            System.out.print("selected product:"+p[n-1]);
        }catch(Exception e){
                System.out.print("Invalid product number");
        }finally{
                    System.out.print("Search completed");

        }
    }
}
```

INPUT

```
1,2,3,4,5
```

OUTPUT

```
Invalid product numberSearch completed
```

Q4. Railway Reservation – Synchronization Debugging

CODE

```
class Main{
    static class Reservation{
        int seats=1;
        synchronized void bookSeat(String name){
            if(seats>0){
                System.out.println(name+"booking a seat");
                seats--;
                System.out.println(name+"booking sucessfully");
            }else{
                System.out.println(name+"=No seat available");
            }
        }
    }
    static class Customer extends Thread{
        Reservation r;
        String name;
        Customer(Reservation r,String name)
        {
            this.r=r;
            this.name=name;
        }
        public void run(){
            r.bookSeat(name);
        }
    }
    public static void main(String[] args)throws Exception{
        Reservation r=new Reservation();
        Customer c1=new Customer(r,"Customer 1");
        Customer c2=new Customer(r,"Customer 2");
        c1.start();
        c2.start();
        c1.join();
        c2.join();
    }
}
```

OUTPUT

```
Customer 1booking a seat
Customer 1booking sucessfully
Customer 2=No seat available
```

Q5. Bank Account – Race Condition Debugging

CODE

```
class Main{
    static class BankAccount{
        int balance=10000;
        synchronized void withdraw(int amount){
            if(balance>=amount){
                System.out.println(Thread.currentThread().getName()+"withdrawing"+amount);
                balance-=amount;
                System.out.println("Withdrawal successful");
            }else{
                System.out.println("Thread.currentThread().getName()+insufficient balance");
            }
        }
    }
    static class ATM extends Thread{
        BankAccount a;
        int amount;
        ATM(BankAccount a,int amount) {
            this.a=a;
            this.amount=amount;
        }
        public void run(){
            a.withdraw(amount);
        }
    }
    public static void main(String[] args)throws Exception{
        BankAccount a=new BankAccount();
        ATM t1=new ATM(a,7000);
        ATM t2=new ATM(a,6000);
        t1.setName("ATM-1");
        t2.setName("ATM-2");
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("Final Balance="+a.balance);
    }
}
```

OUTPUT

```
ATM-2withdrawing6000
Withdrawal successful
Thread.currentThread().getName()+insufficient balance
Final Balance=4000
```